Installation Guide

Flexible Load Balancing — Group 12

1. Prerequisites

The following tools must be installed before running the project:

Tool	Version	Download URL
Java JDK	21 (LTS)	https://adoptium.net
Apache Maven	3.9+	https://maven.apache.org/download.cgi
Docker Desktop	29+	https://www.docker.com/products/docker-desktop
Git	Latest	https://git-scm.com

1.1 Verify Installations

Open a terminal and run the following commands to verify all tools are installed correctly:

```
java -version
mvn -version
docker --version
docker-compose --version
```

Expected output: Each command should return the installed version without errors.

1.2 Environment Variables (Windows)

Set JAVA_HOME to point to your JDK installation:

```
Variable Name: JAVA_HOME
Variable Value: C:\Program Files\Java\jdk-21
Add to PATH: C:\maven\bin and %JAVA_HOME%\bin
```

2. Getting the Project

2.1 Clone from GitHub

Clone the project repository using Git:

```
git clone https://github.com/[your-username]/flexible-load-balancing.git
cd flexible-load-balancing
```

2.2 Project Structure

After cloning, the project should have the following structure:

```
flexible-load-balancing/
■■■ eureka-server/ # Service registry
■ ■■■ src/
■ ■■■ Dockerfile
■ ■■■ pom.xml
■■■ product-service/ # Business microservice
■ ■■■ src/
■ ■■■ Dockerfile
■ ■■■ pom.xml
■■■ gateway-service/ # API gateway
■ ■■■ src/
■ ■■■ Dockerfile
■ ■■■ pom.xml
■■■ docker-compose.yml # Orchestration file
```

3. Building the Project

Build each microservice using Maven. Run these commands from the project root directory:

3.1 Build Eureka Server

```
cd eureka-server
mvn clean package -DskipTests
cd ..
```

3.2 Build Product Service

```
cd product-service
mvn clean package -DskipTests
cd ..
```

3.3 Build Gateway Service

```
cd gateway-service
mvn clean package -DskipTests
cd ..
```

Each build should end with BUILD SUCCESS. If errors occur, ensure Java 21 and Maven are correctly installed.

4. Running with Docker Compose

From the project root directory, run the following command to build Docker images and start all containers:

```
docker-compose up --build
```

This command starts 5 containers: 1 Eureka Server, 3 Product Service instances, and 1 Gateway Service. Wait approximately 2-3 minutes for all services to fully start and register with Eureka.

4.1 Running in Background

To run containers in detached (background) mode:

```
docker-compose up --build -d
```

4.2 Stopping the Project

To stop all running containers:

```
docker-compose down
```

5. Verifying the Installation

After all services have started, verify the installation using the following endpoints:

Endpoint	URL	Expected Result
Eureka Dashboard	http://localhost:8761	Shows 4 registered instances
Gateway — Products	http://localhost:8080/api/products	Returns product list + instance info
Product Instance 1	http://localhost:8081/products	Returns from product-instance-1
Product Instance 2	http://localhost:8082/products	Returns from product-instance-2
Product Instance 3	http://localhost:8083/products	Returns from product-instance-3

5.1 Testing Load Balancing

To verify load balancing is working, send multiple requests to the gateway endpoint and observe that the 'instance' field in the response changes:

```
curl http://localhost:8080/api/products
```

Repeat the above command several times. Each response should show a different instance (product-instance-1, product-instance-2, or product-instance-3), confirming that requests are being distributed across all available instances.

6. Troubleshooting

Problem	Solution
Docker daemon not running	Open Docker Desktop and wait for "Engine running" status
BUILD FAILURE during mvn	Ensure JAVA_HOME is set correctly to JDK 21
503 Service Unavailable	Wait 2-3 minutes for all services to register with Eureka
Port already in use	Stop any other applications using ports 8080, 8081-8083, 8761
No instances in Eureka	Run docker-compose down then docker-compose up --build again

For further assistance, contact the project team via GitHub repository.